Algorithmics	Student information	Date	Number of session
	UO: 300737	03/03/2025	3
	Surname: Hoyos		
	Name: Ignacio		

Escuela de Ingeniería Informática
Universidad de Oviedo

Universidad de Oviedo
Universidá d'Uviéu
University of Oviedo

Activity 1: Divide and Conquer by subtraction

Subtraction 1 -> a=1, b=1, k=0 -> O(n)

Subtraction 2 -> a=1, b=1, k=1 -> O(n^2)

Subtraction 3 -> a=2, b=1, k=0 -> O(2^n)

For what value of n do the Subtraction1 and Subtraction2 classes stop giving times (we abort the algorithm because it exceeds 1 minute)? Why does that happen?

The subtraction 1 and 2 stop because of a StackOverFlowError when n>`8192`.

How many years would it take to complete the Subtraction3 execution for n=80? Reason the answer?

Taking into account that for n=30, it takes 38550 ms, so for n=80, it will take 38550*2^50 = 4.34*10^19 ms that are equivalent to **1 375 372 100 000 years.**

TABLE1 (times in milliseconds WITHOUT OPTIMIZATION)

n	Substraction4
100	LoR
200	LoR
400	82
800	630
1600	4 867
3200	39 446
6400	OoT

Algorithmics	Student information	Date	Number of session
	UO: 300737	03/03/2025	3
	Surname: Hoyos		
	Name: Ignacio		

TABLE2 (times in milliseconds WITHOUT OPTIMIZATION)

n	Substraction5
30	360
32	1 112
34	3 355
36	9 959
38	29405
40	OoT

How many years would it take to complete the Subtraction5 execution for n=80? Reason the answer?

Taking into account that for n=38, it takes 29405ms, so for n=80, it will take 29405ms *3^51 = 6.3329371e+28 ms that are equivalent **2.0067867 *10^21 years.**

Activity 2: Divide and conquer by division

Division 1 -> a=1, b=3, k=1 -> O(n)

Division 2 -> a=2, b=2, k=1 -> O(n*log(n))

Division 3 -> a=2, b=2, k=0 -> O(n^log2(2)) ->O(n)

TABLE3 (times in milliseconds WITHOUT OPTIMIZATION)

n	Division4	Division5
1000	LoR	LoR
2000	LoR	96
4000	65	373
8000	1 019	1 508
16000	4 094	6 075
32000	16 442	23 989
64000	OoT	OoT

Algorithmics	Student information	Date	Number of session
	UO: 300737	03/03/2025	3
	Surname: Hoyos		
	Name: Ignacio		

Activity 3: Two basic examples

VectorSum 1 -> Iterative -> O(n)

VectorSum 2 -> a=1, b=1, k=0 -> O(n)

VectorSum 3 -> a=2, b=2, k=0 -> O(n)

Fibonacci 1 -> Iterative -> O(n)

Fibonacci 2 -> Iterative -> O(n)

Fibonacci 3 -> a=1, b=1, k=0 -> O(n)

Fibonacci 4 -> a=1, b=2 or b=1, k=1 -> O(2^n) - O(n^1) -> O(1.6^n)

TABLE4 (times in milliseconds WITHOUT OPTIMIZATION)

n	VectorSum	VectorSum 2	VectorSum 3
3	0,0000398	0,000074	0,000087
6	0,0000627	0,000122	0,000162
12	0,0000868	0,000236	0,000354
24	0,000137	0,000429	0,000734
48	0,000228	0,000818	0,001457
96	0,000415	0,001632	0,002889
192	0,000781	0,003173	0,005773
384	0,00152	0,006348	0,011607
768	0,00297	0,012695	0,024014
1536	0,00592	0,025385	0,0510
3072	0,0119	0,055	0,0984
6144	0,0236	0,108	0,1973
12288	0,047	Stack Overflow	0,3826
24576	0,095	Stack Overflow	0,7845
49152	0,188	Stack Overflow	1,5629
98304	0,379	Stack Overflow	3,1487

Although they have the same complexity (O(n)), which can be seen is true as its response time's doubles when the size of n doubles. They differ mostly in the initial value, which is probably due to the recursive calls, it makes sense that call and call and call again a method instead of doing directly increase the execution time. And it makes sense as sum1 which is iterative is the fastest, sum2 which have a recursive call is slower and sum3 which have two recursive calls is the slowest.

TABLE5 (times in milliseconds WITHOUT OPTIMIZATION)

n	Fibonacci 1	Fibonacci 2	Fibonacci 3	Fibonacci 4
10	0,000091	0,000114	0,000182	0,00233
11	0,000089	0,000114	0,000211	0,00361
12	0,000091	0,000124	0,000224	0,00583
13	0,000097	0,000126	0,000244	0,00959
14	0,000099	0,000133	0,000254	0,01574
15	0,000104	0,000140	0,000270	0,02495
20	0,000122	0,000185	0,000347	0,276
25	0,000150	0,000210	0,000437	3,2
30	0,000166	0,000244	0,000502	32,81
35	0,000188	0,000276	0,000580	374
40	0,000209	0,000318	0,000657	4187
45	0,000228	0,000350	0,000743	46848
50	0,000250	0,000379	0,000819	OoT
55	0,000271	0,000416	0,000913	OoT
59	0,000301	0,000441	0,000948	OoT

This reaffirm what I suggested in the previous table, the more recursive calls, the more time taken, Fib 1 which is iterative is the fastest, Fib 3 that contains one recursive call is

slower and Fib 4 which contains two recursive calls is the slowest by far (although it has a different complexity than the others). Thinking about Fib 4, it have even more sense, in the Divide and Conquer algorithms, increasing the variable a (number of sub problems) means increase the time complexity. And also there is other thing to discuss, why Fib 2 is slower than Fib 1? Probably because of using vector positions instead of integers.

Activity 4: Petanque championship organization

TABLE6 (times in milliseconds WITHOUT OPTIMIZATION)

n	Fibonacci 1
2	0 (LoR)
4	0 (LoR)
8 (1)	0 (LoR)
16	23 (LoR)
32	17 (LoR)
64	38 (LoR)
128	36 (LoR)
256	66
512	49 (LoR)
1024	76
2048	72
4096	108
8192	Stack OverFlow

A StackOverflowError would occur if the number of recursive calls exceeds the JVM's maximum stack size, that is when competing more than 2^12 (4096) participants.